Computer Science & IT



Data Structure & Programming

Linked List

Lecture No. 04



By- Abhishek Sir

Recap of Previous Lecture



- Topic — Insert begin
- Topic — Insert end
- Topic — Delete (2 pointer traversal)
- Topic — practice
- Topic

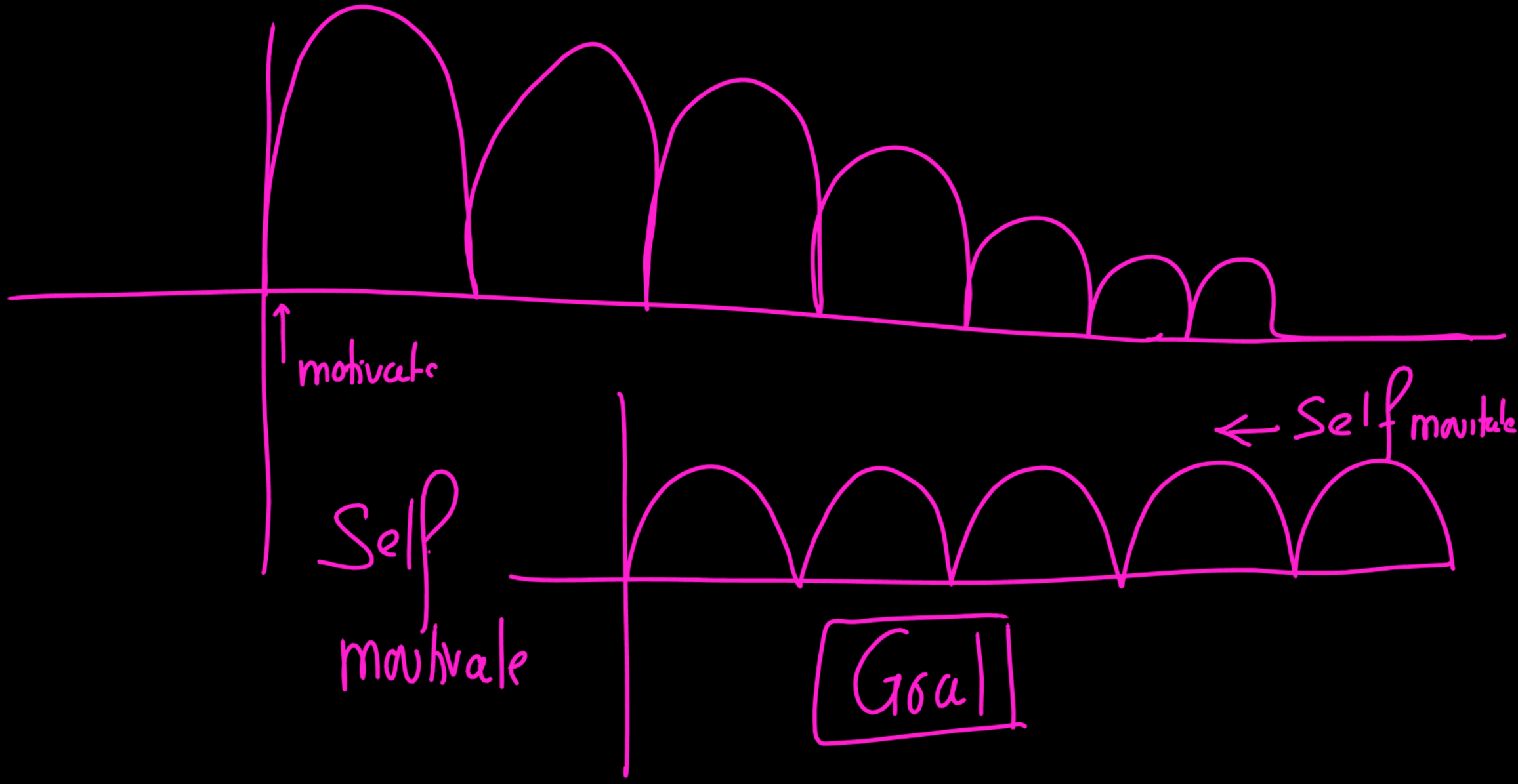
Slide

Topics to be Covered



- Topic — Insert after given Node
- Topic — Reverse Linked List
- Topic — find middle of Linked List ✓
- Topic — Double Linked List ✓
- Topic

Slide


motivate
Self
motivate
Self motivate
Goal



Topic : Single Linked List



Slide

Topic Question

If the list contain the value 1→2→3 then what is the Output following function.

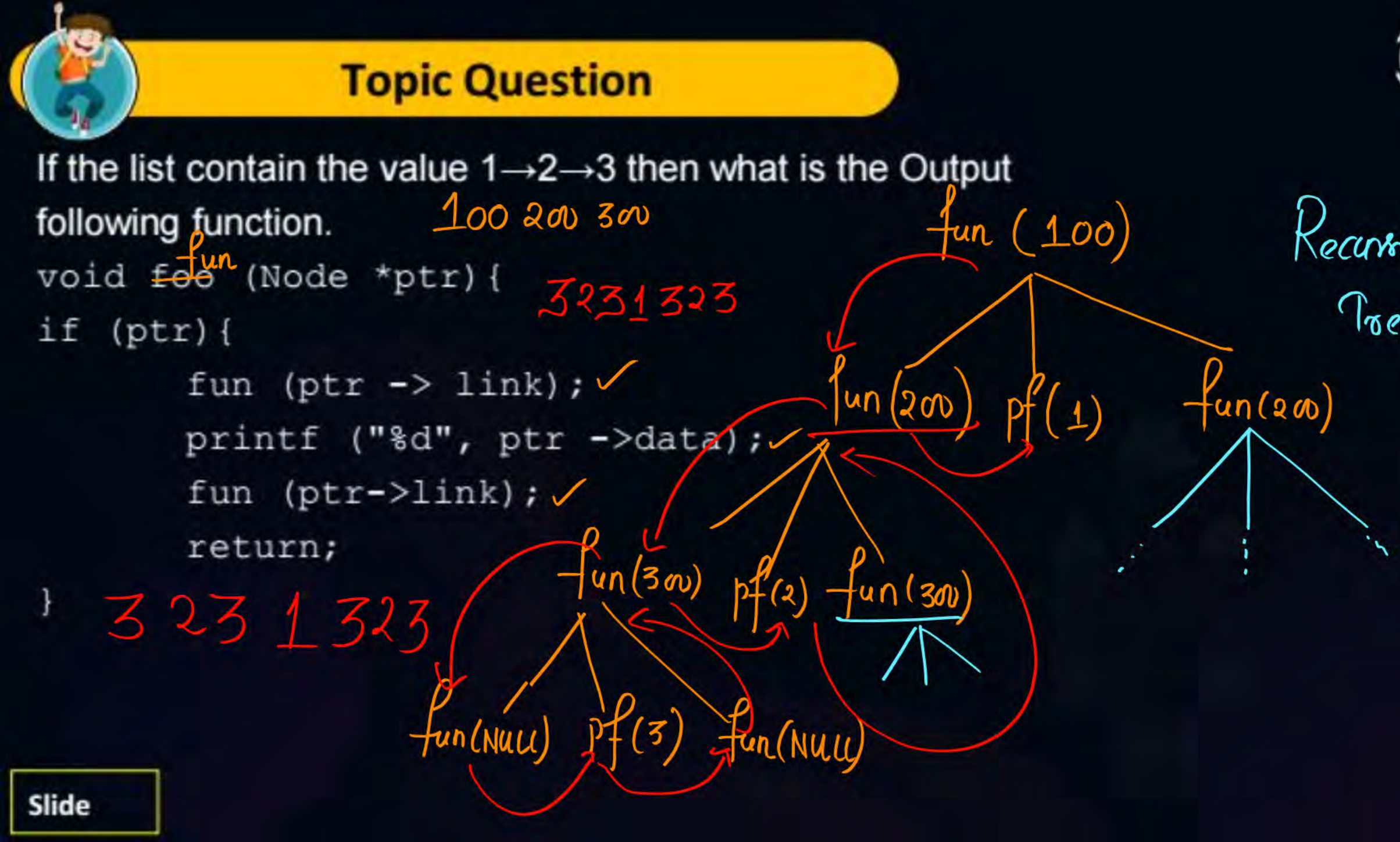
```
void foo (Node *ptr){
if (ptr){
        fun (ptr -> link);
        printf ("%d", ptr ->data);
        fun (ptr->link);
        return;
}
```

100 200 300

fun

3231323

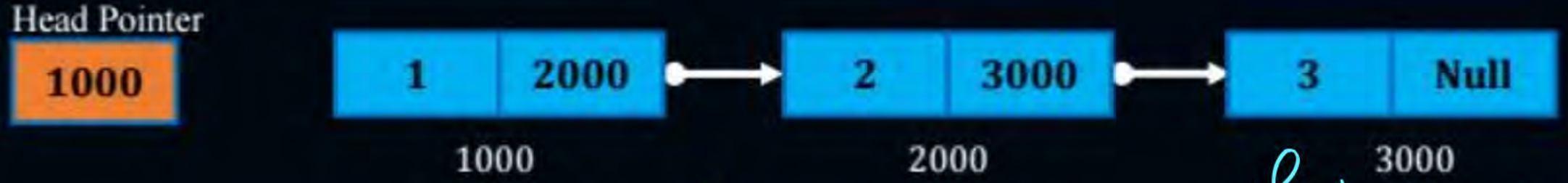
3 23 1 323



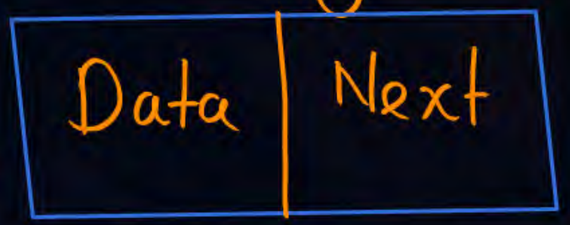
Slide



Topic : NULL Pointer Dereferencing



Node * ptr = getnode();



ptr | 100

```
typedef struct node{
    int data;
    struct node * next;
} Node;
```

ptr | NULL

if ptr NULL

ptr → data ??

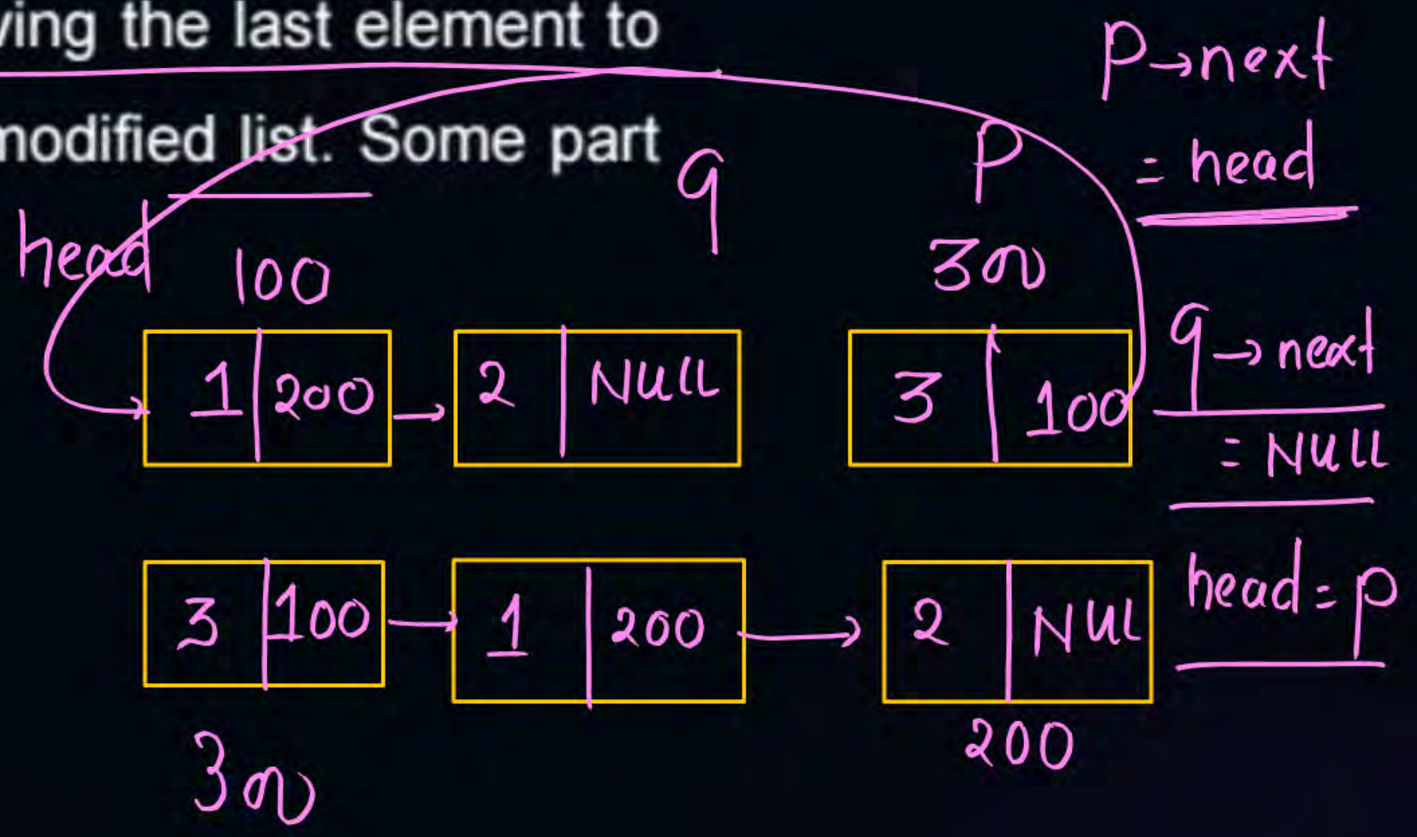
ptr → next;

NULL pointer dereference

Slide

The following C function takes a ~~simply~~ Single-linked list as input argument. It modifies the list by moving the last element to the front of the list and returns the modified list. Some part of the code is left blank.

```
typedef struct node {
        int value;
        struct node *next;
}Node;
```



```
Node *move to front (Node *head) {
    Node *p, *q;
    if ((head == NULL) || (head -> next = = NULL))
        return head;
    q= NULL; p=head;
    while (p-> next! = NULL) {
        q = p;
        p = p -> next;
    }
    _____
    return head;
```

1 Node

Empty

2 pointer Traversal

Choose the correct alternative to replace the blank line.

(A) `q = NULL; p - > next = head; head = p;`

(B) `q -> next = NULL; head = p; p -> next = head;`

(C) `Head = p; p -> next = q;q -> next = NULL;`

(D) `q -> next = NULL; p -> next = head; head = p;`

Topic Question

The following C function takes a single-linked list of integers as a parameter and rearranges the element of the list. The function is called with the list containing the integers 1, 2, 3, 4, 5, 6, 7 in the given order. What will be the contents of the list after the function completes execution?

```
struct node {
        int value;
        struct node *next;
};
```

Topic Question

```
void rearrange (struct node *list){
strucut node *p, *q;
int temp;
if (!list || !(list-> next)) return;
p = list; q = list -> next;
while (q) {
temp = p ->value; p -> value = q -> value;
q -> value = temp; p = q ->next;
q = p ? p-> next: 0;


}
}
```

(A) 1, 2, 3, 4, 5, 6, 7
(B) 2, 1, 4, 3, 6, 5, 7
(C) 1, 3, 2, 5, 4, 7, 6
(D) 2, 3, 4, 5, 6, 7, 1

H.W

1 → 2 → 3 → 4 → 5 → 6 → 7

Question

What is output of the program if the list contains 1→2→3?

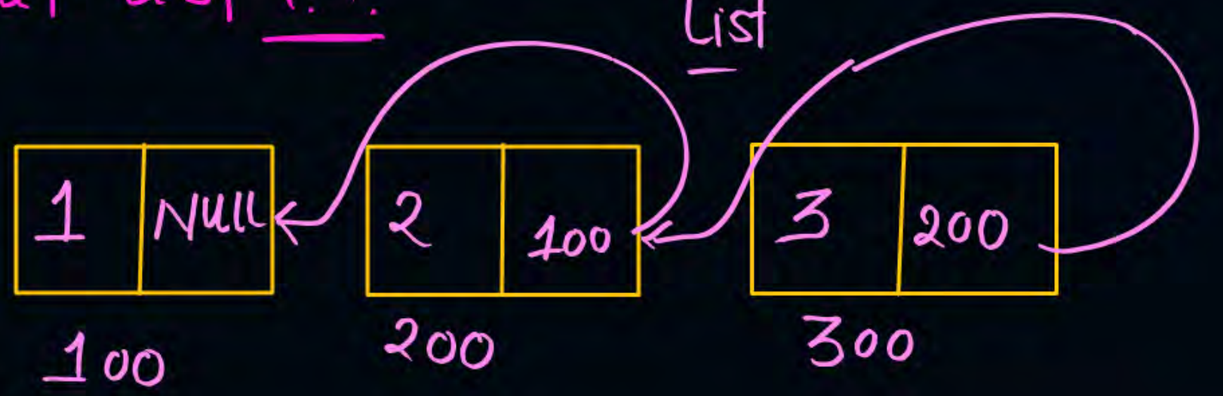
```
HP is global variable
void Dosomething(){
        Node *q, *r, *s;
        q = HP;
        r = NULL;
              while( q!=NULL){
                    s = r;
                    r = q;
                    q = q→ link;
                    r→ link =s;
              }
              HP = r;
}
```

3→2→1

output list ??

Reverse of Linked List

HP = 300



r: NULL

q

s	r	q
	s	r
		q
s	r	
s -		r
	q	
	r	NULL
		q :



Topic : Single Linked List

Let's understand the problem!

Given a singly linked list, write a program to find the middle node of the linked list. If the number of nodes is even, we need to return the second middle node.

Example 1: Input: 5->4->3->2->1, Output: 3

Explanation: Here the number of nodes is 5, so there is one middle node which is 3.

Example 2: Input: 6->5->4->3->2->1, Output: 3

Explanation: The number of nodes is 6, where the first middle node is 4 and the second middle node is 3. So we need to return the pointer to the node 3.

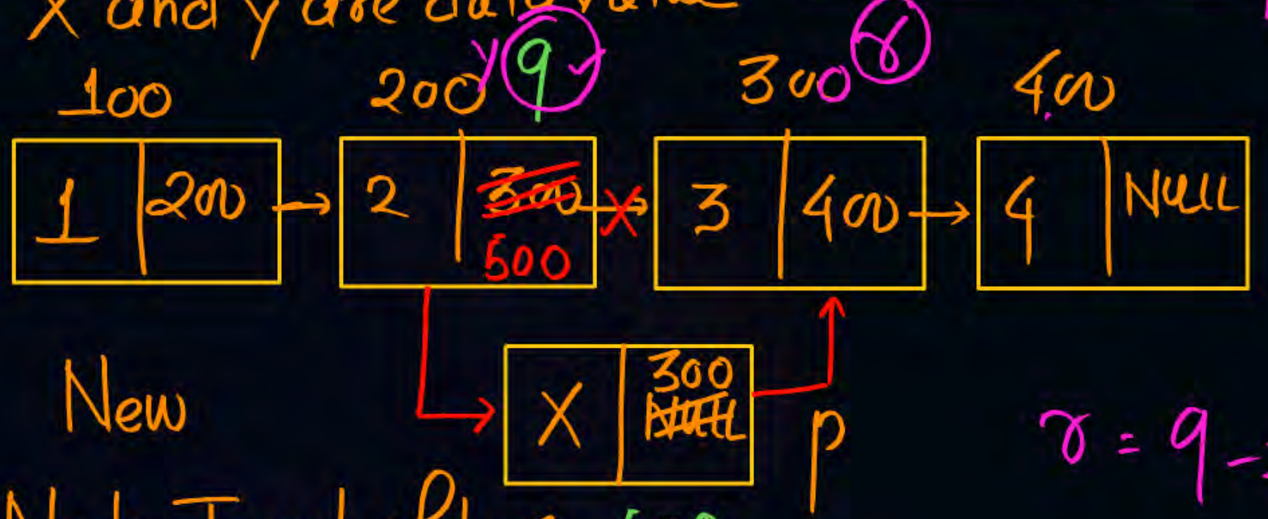
Topic : Single Linked List

Insert x after y

```
q->next = p
p-> next = r
```

HP is global

* X and y are data value



r = q->next

Node Insert after 500

Data 2

Slide



Topic : Single Linked List

```
void AddxofteoY( int x, int y){
    Node * p = getnode(x);
    Node* q = HP, *r
    if(HP == NULL){ // Empty list
        free(p);
        return;
    }
    else{
        while( q->data != y && q->next != NULL)
            q = q->next;
        if( q->data == y){
            r = q->next;
            q->next = p;
            p->next = r;
        }
        else{
            free(p);
            return;
        }
    }
}
```

Last Node

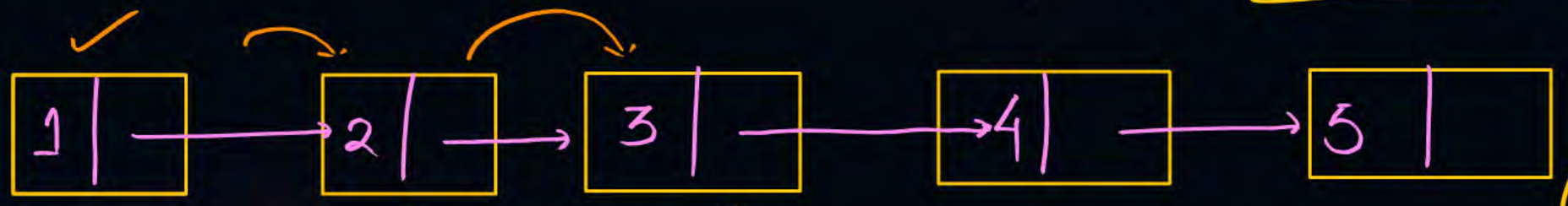
Slide



Topic : Single Linked List

Return middle element List

$\lceil 5/2 \rceil = \lceil 2.5 \rceil = 3$



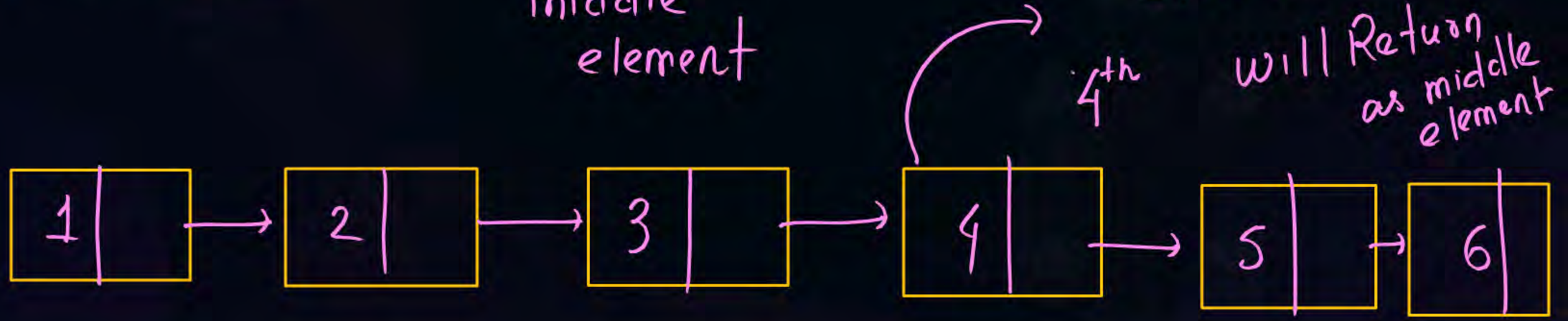
i = 1

3+1 = 4 odd No.

$(2/2 + 1)$

Even No.

middle element



4th will Return as middle element

Slide



Topic : Single Linked List

```
int count (Node *s){
    int counter=0;
    while(S!=NULL){
        counter ++;
        S=S->next;
    }
    return counter
}
```

→

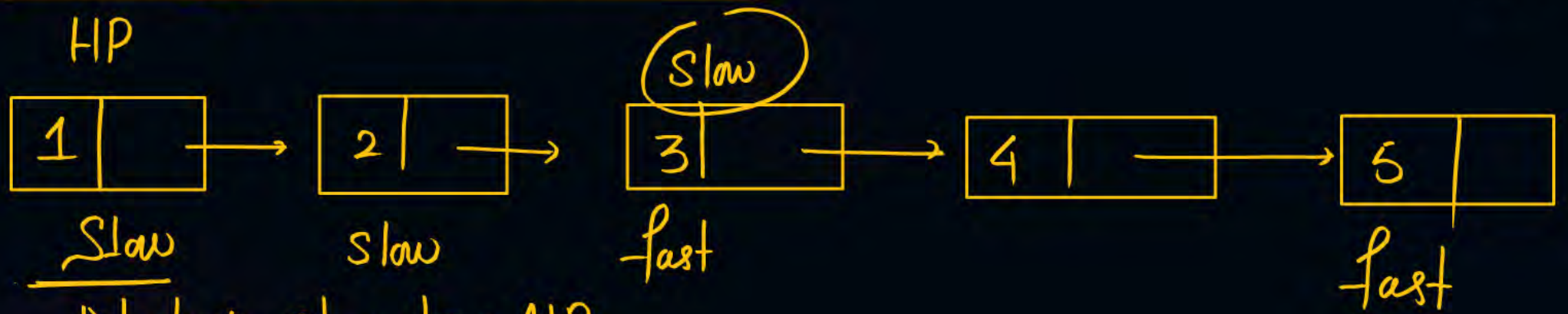
HP - global

```
int middleLL( ){
  int i=1;
  int length= count(HP); Node*q=HP;
 if (length>0){
 while (i< length/2 +1) {
        q=q->next;
        i++;
    }
    return q->data;
 }
 return -1;
}
```

Slide



Topic : Single Linked List



Node* slowptr = HP;

Node* fastptr = HP;

slowptr = slowptr → next;

fastptr = fastptr → next → next;

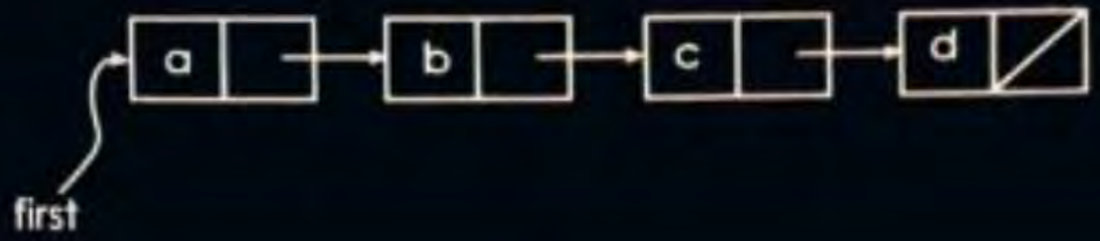
Concept

Next class it will complete

Slide

Topic : Question

What does Do( ) do on a singly linked list




```
int Do (struct node *p)
{
        if (p)
            return 1 + Do (p → link);
        else
            return 0;
}
void main ( )
{
        printf ("%d", Do (first));
}
```

Slide

(a) 1 (b) 2 (c) 3 (d) 4



2 mins Summary

Topic — Insert x after y

Topic — Reverse of List

Topic — middle element

Topic — Move to front

Topic

Slide

THANK - YOU

Slide